

A Review of Computer-Use Agents in Real Operating Systems

From Agent Frameworks to Open Foundations and
Tool-Augmented Evaluation

Yaoyao Xu

224040357

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Computer-use agents (CUAs) have recently emerged as a major research direction at the intersection of multimodal learning, agent systems, and operating-system-level automation. Unlike web-only agents or pure API-calling assistants, CUAs are expected to perceive arbitrary desktop states, reason over long-horizon goals, and act through the same graphical interfaces that humans use. This report reviews three representative papers that together capture a coherent evolution of this area: *Agent S: An Open Agentic Framework that Uses Computers Like a Human*, *OpenCUA: Open Foundations for Computer-Use Agents*, and *OSWorld-MCP: Benchmarking MCP Tool Invocation in Computer-Use Agents*. The three papers are analyzed under a shared research lens: framework design, open infrastructure, and benchmark design.

The report argues that these works should not be read as isolated contributions. Agent S addresses the question of how to build a stronger computer-use agent by combining hierarchical planning, retrieval from web knowledge, and experience-based memory. OpenCUA moves the field to a new layer by building open data collection tools, large-scale training datasets, reflective reasoning supervision, and competitive open-source agent models. OSWorld-MCP then updates the evaluation layer by showing that benchmarking GUI interaction alone is no longer sufficient once agents can invoke external tools through the Model Context Protocol (MCP). By jointly reviewing these papers, the report identifies a broader research trajectory: progress in real-OS agents requires the co-design of capabilities, foundations, and measurement.

Beyond summarizing each paper, the report provides a comparative analysis of their goals, technical mechanisms, empirical findings, strengths, and limitations. It concludes that current computer-use agents still face substantial challenges in grounding, long-horizon robustness, hybrid decision making, and evaluation stability, but the reviewed papers establish a strong foundation for future systems research on agentic interaction in real operating systems.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Research Scope and Questions	1
1.3 Central Thesis of the Report	2
1.4 Report Structure	2
2 Background and Related Work	3
2.1 What Is a Computer-Use Agent?	3
2.2 Why Real Operating Systems Are Hard	3
2.3 Representative Prior Directions	4
2.4 Analytical Dimensions Used in This Report	5
2.5 A Shared Evolutionary Line	5
2.6 Summary	5
3 Agent S: A Framework for Human-Like Computer Use	7
3.1 Research Problem and Positioning	7
3.2 Framework Overview	7
3.3 Experience-Augmented Hierarchical Planning	8
3.4 Memory Construction and the Agent-Computer Interface	9
3.5 Experimental Setup and Main Results	10
3.6 Ablations and Error Analysis	11
3.7 Assessment	11
3.8 Summary	11

4	OpenCUA: Open Foundations for Computer-Use Agents	12
4.1	Research Problem and Contribution Scope	12
4.2	AgentNet Tool and Dataset Construction	12
4.3	Reflective Long-CoT Supervision	13
4.4	Empirical Results and Scaling Laws	14
4.5	Why OpenCUA Matters	15
4.6	Summary	16
5	OSWorld-MCP: Benchmarking GUI and Tool Invocation Together	17
5.1	Research Problem and Benchmark Motivation	17
5.2	From GUI-Only Tasks to Hybrid Action Spaces	17
5.3	Tool Generation and Benchmark Construction	18
5.4	Metrics: Accuracy, TIR, and ACS	19
5.5	Main Results	20
5.6	Assessment	20
5.7	Summary	21
6	Comparative Synthesis and Future Directions	22
6.1	From Frameworks to Foundations to Evaluation	22
6.2	Direct Comparison of the Three Papers	22
6.3	Common Trends	22
6.4	Outstanding Limitations	24
6.5	Future Directions	24
6.6	Summary	25
7	Conclusion	26
7.1	Summary of Contributions	26
7.2	Future Work	26

List of Figures

3.1	Agent S uses a computer like a human to solve diverse desktop tasks. . .	8
3.2	Framework overview of Agent S.	8
3.3	Memory construction and continual update in Agent S.	10
4.1	Overview of the OpenCUA framework.	13
4.2	Reflective long-CoT synthesis in OpenCUA.	14
4.3	Data scaling curves across benchmarks in OpenCUA.	15
5.1	A task completed through GUI operations versus MCP tool invocation. .	18
5.2	Overview of the OSWorld-MCP framework.	19
5.3	Automated tool generation and wrapping in OSWorld-MCP.	19
5.4	Relationships among TIR, accuracy, and average completion steps in OSWorld-MCP.	20

List of Tables

2.1	Analytical dimensions used in this report.	5
3.1	Selected Agent S results on OSWorld.	10
4.1	Selected OpenCUA results on OSWorld-Verified.	14
5.1	Selected OSWorld-MCP results.	20
6.1	Comparison of the three reviewed papers.	23

Chapter 1

Introduction

1.1 Motivation

Large language models and multimodal foundation models have made it increasingly plausible to build agents that operate general-purpose software environments. Earlier generations of automation systems mostly relied on brittle macros, manually written scripts, or narrow task-specific policies. In contrast, recent computer-use agents aim to interpret arbitrary user instructions, observe the current screen, reason about intermediate goals, and then interact with software through mouse and keyboard operations. This shift is important because it treats existing software interfaces as the primary execution substrate rather than assuming privileged access to application internals.

From an operating-systems perspective, this topic is compelling for three reasons. First, the environment is real rather than synthetic: the agent must function across heterogeneous applications, changing windows, textual and visual content, and long chains of dependent actions. Second, the control loop is multimodal and interactive: perception, planning, execution, and feedback are tightly coupled. Third, the evaluation problem is nontrivial because success depends not only on what the agent knows but also on how it acts under dynamic interface constraints. These characteristics make CUAs a systems problem, not merely a prompting or model-scaling problem.

1.2 Research Scope and Questions

This report focuses on three recent papers that together define a coherent research focus:

1. How can one design a stronger framework for using computers like a human?
2. What open data and model foundations are needed for scalable and reproducible CUA research?

3. How should evaluation evolve when agents can choose between GUI interaction and external tool invocation?

These questions are respectively addressed by Agent S [2], OpenCUA [22], and OSWorld-MCP [12]. Although each paper emphasizes a different layer of the stack, together they form a progression from capability engineering to infrastructure building and then to benchmark redesign.

1.3 Central Thesis of the Report

The main claim advanced in this report is that the reviewed papers collectively describe an architectural evolution of computer-use agents in real operating systems. Agent S shows that long-horizon task success can be improved through explicit planning and experience retrieval. OpenCUA shows that such improvements cannot scale in a reliable or reproducible way without open annotation tools, large datasets, and principled training recipes. OSWorld-MCP shows that once agents gain access to external tools, benchmark design must also change, because GUI-only evaluation underestimates an increasingly important dimension of competence. In short, the field is moving from *building stronger agents*, to *building open foundations*, to *measuring hybrid capabilities fairly*.

1.4 Report Structure

The rest of the report is organized as follows. Chapter 2 defines the computer-use agent setting and introduces the analytical framework used throughout the review. Chapters 3, 4, and 5 examine Agent S, OpenCUA, and OSWorld-MCP in detail. Chapter 6 synthesizes the three papers, identifies shared trends and unresolved limitations, and outlines future directions. Chapter 7 concludes the report.

Chapter 2

Background and Related Work

2.1 What Is a Computer-Use Agent?

A computer-use agent is an AI system that receives a natural-language goal, perceives the current state of a software environment, and completes the task by issuing interface-level actions such as clicking, typing, scrolling, dragging, or invoking higher-level tools. The defining property of this class of agents is that it operates *through* the interface rather than by bypassing it.

This makes CUAs distinct from several neighboring paradigms. Web agents often rely on browser-centric abstractions such as the DOM, which can simplify grounding and action generation. Representative web-agent benchmarks and environments include Mind2Web, WebArena, VisualWebArena, and WorkArena [7, 8, 15, 33]. Tool-use LLMs can solve tasks by invoking structured functions or APIs directly, often without needing to reason about visual interface state, as in ReAct-style or Toolformer-style setups [29, 30]. Traditional GUI automation scripts can replay fixed action sequences efficiently, but they typically fail when interfaces change or when task instructions vary. CUAs combine the open-endedness of language agents with the low-level control requirements of interface automation.

2.2 Why Real Operating Systems Are Hard

The three papers reviewed in this report all start from a shared observation: operating real desktop environments is difficult even for strong multimodal models. Several sources of difficulty recur across the literature.

First, perception and grounding remain fundamental bottlenecks. Real screens contain dense mixtures of text, icons, menus, dialogs, and partially occluded elements. The agent must distinguish relevant from irrelevant content while mapping abstract instructions to precise targets. Second, many tasks are long-horizon and involve interdependent steps.

An agent may need to switch across applications, maintain intermediate state, recover from mistakes, and decide when a subgoal has been completed. Third, software interfaces are dynamic and non-uniform. Unlike environments with fixed APIs, desktop agents face constant variation in layout, software version, interaction pattern, and feedback mechanism.

Fourth, modern computer-use settings increasingly mix GUI actions with tool invocation. When an external tool can complete in one step what would otherwise require several fragile GUI manipulations, the agent faces a hybrid decision problem: it must choose not only *what* to do but also *how* to do it. Finally, evaluation itself is challenging. Success depends on path quality, environment setup, step budget, and often on subtle task-specific conditions. Benchmarks therefore shape research conclusions in unusually strong ways.

2.3 Representative Prior Directions

The recent literature around computer-use agents can be grouped into four directions. The first direction is *interactive environments and agent benchmarks*. OSWorld and Windows Agent Arena are the most directly relevant real-desktop settings for the three papers reviewed here [6, 27]. Related benchmark efforts include OmniAct, AndroidWorld, and OSWorld-Human [1, 13?]. These works establish that realistic agent evaluation requires executable tasks and dynamic feedback, not only static screenshots or action logs.

The second direction is *GUI grounding and action modeling*. ShowUI, UGround, OS-Atlas, ScreenSpot-Pro, and UI-Vision all study how language should be grounded onto interface elements or action targets [10, 17, 18, 21, 26]. They are highly relevant because grounding failures remain one of the most common sources of agent error. However, these resources alone do not solve long-horizon task execution.

The third direction is *end-to-end computer-use systems*. Representative works include Aguis, UI-TARS, Mobile-Agent, Mobile-Agent-v3, Agent S2, CoAct, and ComputerRL [3, 16, 24, 28, 31? , 32]. These systems differ in how they represent actions, whether they use code as an action channel, whether they emphasize multi-agent collaboration, and whether they rely on supervised fine-tuning or online reinforcement learning. Together they show that the field is moving from simple single-step visual grounding toward longer, more adaptive workflows.

The fourth direction is *tool-use protocols and benchmark evolution*. MCP has become an important interoperability standard for connecting agents to external tools [4]. Benchmarks such as MCPEval, MCP-Radar, LiveMCPBench, and MCP-Bench show that the community is beginning to treat tool invocation as a first-class evaluation problem [9, 19, 20, 23]. OSWorld-MCP belongs to this line of work but differs by placing tool-use

Table 2.1: Analytical dimensions used in this report.

Dimension		Questions considered in the review
Problem	formulation	What concrete research bottleneck is the paper trying to solve?
Technical	mechanism	What system architecture, data pipeline, or benchmark protocol is proposed?
Empirical evidence		What results, ablations, or case studies support the claims?
Broader	contribution	How does the work change the research landscape for CUAs?

decisions inside a realistic GUI-based operating environment.

2.4 Analytical Dimensions Used in This Report

To compare the three papers consistently, this report uses four analytical dimensions: problem formulation, technical mechanism, empirical evidence, and broader research contribution. Problem formulation asks what specific gap the paper identifies. Technical mechanism examines the system or benchmark design proposed to address that gap. Empirical evidence focuses on reported evaluation results, ablations, and analysis. Broader contribution considers how the paper changes the field beyond its immediate numbers.

2.5 A Shared Evolutionary Line

The papers can be placed on a single research line. Agent S addresses the *framework layer*: it asks how to make an agent better at acting in real GUI environments. OpenCUA addresses the *foundation layer*: it asks how to create open data, tooling, and training recipes so that such agents can be studied and scaled systematically. OSWorld-MCP addresses the *evaluation layer*: it asks how to benchmark agents fairly once GUI operation is no longer the only meaningful action channel. This evolutionary line will be revisited in the synthesis chapter.

2.6 Summary

The broader related-work landscape shows that computer-use agents sit at the intersection of web agents, GUI grounding, multimodal planning, and tool-using agent systems. The three reviewed papers stand out because together they define a connected response to the next major challenge in the field: making real-OS agents stronger, more open, and

more fairly evaluated at the same time.

Chapter 3

Agent S: A Framework for Human-Like Computer Use

3.1 Research Problem and Positioning

Agent S [2] addresses a practical and immediately visible problem in early desktop agents: even when powerful multimodal models are available, success rates on realistic OS tasks remain low. The paper argues that this underperformance is driven by three interacting limitations. First, the agent often lacks domain-specific and up-to-date procedural knowledge about applications or websites. Second, it struggles with long-horizon planning because many OS tasks require multi-step sequencing with intermediate dependencies. Third, it faces unstable grounding in dynamic and non-uniform interfaces, where translating high-level intentions into concrete actions is error-prone.

The paper’s response is to treat GUI automation as an explicitly structured agent problem. Rather than relying on a single end-to-end decision loop, Agent S introduces a framework that mixes planning, retrieval, and constrained interface control. Its key contribution is not merely a new prompt but a systems-level architecture that decomposes control into distinct modules.

3.2 Framework Overview

Figure 3.1 illustrates the high-level ambition of Agent S: the agent uses a computer through the same interface channels as a human user, but augments its behavior with web knowledge, internal memories, and hierarchical planning. This framing is important because it positions computer-use not as a narrow benchmark trick but as a general interaction capability.

The main operational structure of Agent S is shown in Figure 3.2. A manager mod-

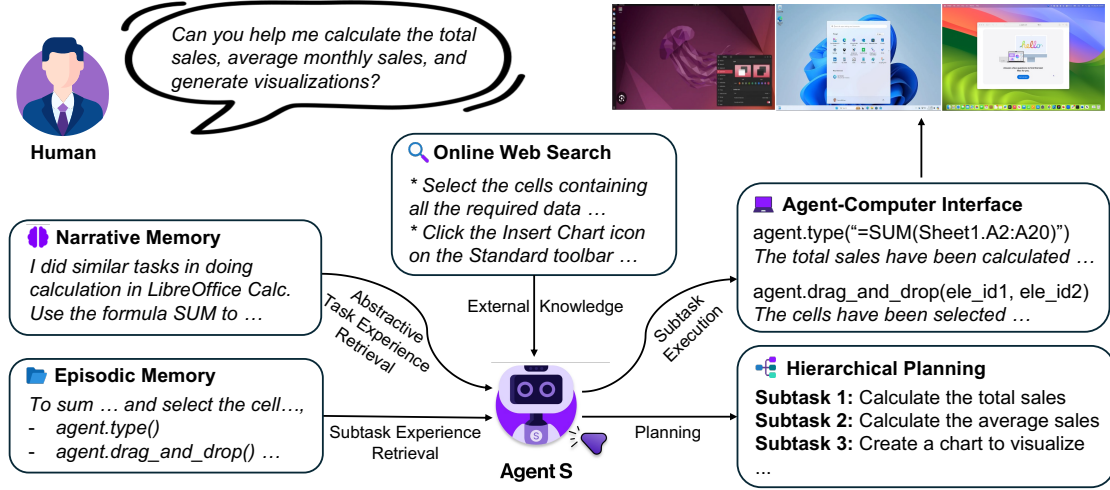


Figure 3.1: Agent S uses a computer like a human to solve diverse desktop tasks.

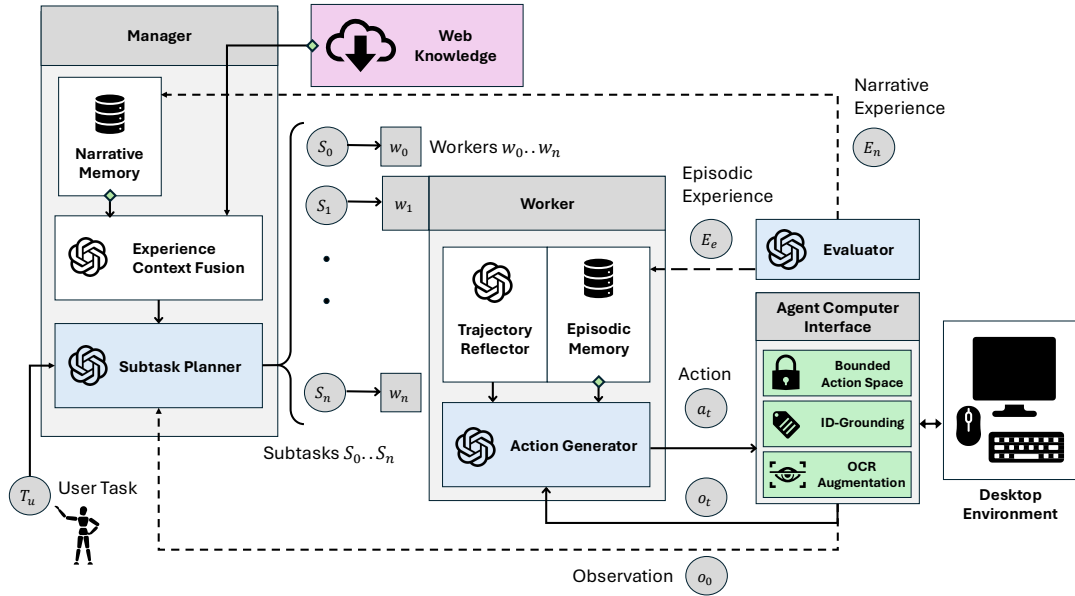


Figure 3.2: Framework overview of Agent S.

ule receives the user task and initial observation, retrieves both external and internal knowledge, and decomposes the task into subtasks. Worker modules then execute these subtasks sequentially. A self-evaluator summarizes successful or failed experiences and writes them back into memory. The result is a closed loop in which planning and execution can be improved over time through stored experience.

3.3 Experience-Augmented Hierarchical Planning

The central technical idea in Agent S is *experience-augmented hierarchical planning*. The manager first formulates a query from the user goal and the current interface state. This query is used in two retrieval channels. One channel accesses online web knowledge, which

provides application-specific or procedure-specific information that may not be reliably present in the base model. The other accesses narrative memory, which stores high-level summaries of prior task trajectories. The retrieved evidence is fused and used to generate a topologically ordered queue of subtasks with contextual hints.

This design is important because it distributes knowledge use across levels of abstraction. General procedural hints from the web help when the agent needs up-to-date instructions about unfamiliar software behavior. Past task summaries help the manager reason about coarse-grained strategies. Subtask execution then becomes more local and more controllable than a flat end-to-end action sequence.

Worker modules add another layer of experience use. For each subtask, the worker retrieves relevant entries from episodic memory, which stores more detailed subtask-level summaries. A trajectory reflector monitors the subtask execution process and provides guidance intended to reduce repetitive actions or poor local decisions. The worker ultimately outputs grounded primitive actions through the interface abstraction discussed below.

3.4 Memory Construction and the Agent-Computer Interface

Agent S distinguishes between two memory types. *Narrative memory* stores high-level task summaries that are useful for planning. *Episodic memory* stores detailed subtask experience that is useful for execution. This separation is conceptually well motivated: planning and execution operate at different temporal and semantic scales, so the knowledge representation need not be the same for both.

The paper also describes how these memories are built. Initial experience is collected through self-supervised exploration over synthetic tasks, and both memories continue to be updated during later interaction. Figure 3.3 presents this pipeline. The continual update mechanism matters because GUI environments are open-world and non-stationary; a memory that never changes would quickly lose value.

The second major technical component is the Agent-Computer Interface (ACI). The paper argues that desktop interfaces are built for two extremes: humans, who have rich perception but no direct programmatic execution channel, and traditional software, which has APIs or scripts but little visual understanding. Multimodal agents fall between these extremes. Agent S therefore introduces a language-centric abstraction that combines screenshot input with an image-augmented accessibility tree and constrains actions to a bounded set of language-level primitives. This design attempts to improve safety, grounding precision, and the temporal granularity of feedback.

From a systems perspective, ACI is one of the most significant ideas in the paper.

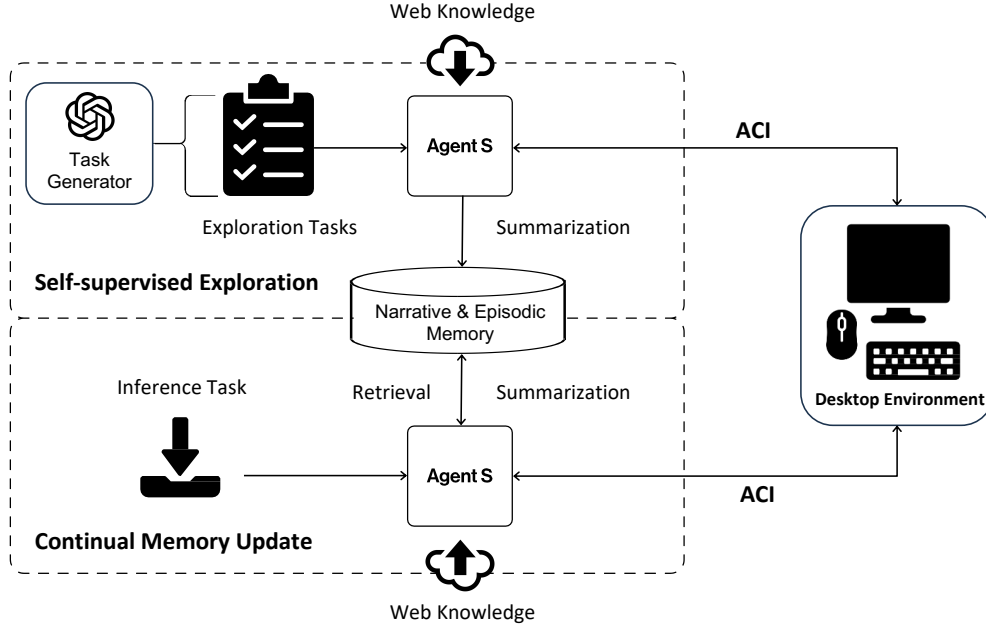


Figure 3.3: Memory construction and continual update in Agent S.

Table 3.1: Selected Agent S results on OSWorld.

Method	Daily	Professional	Overall
GPT-4o baseline	12.33	14.29	11.21
Agent S w/ GPT-4o	27.06	36.73	20.58
Agent S w/ Claude-3.5	30.46	32.65	20.48

It makes explicit the interface contract between the foundation model and the operating environment. Rather than allowing unconstrained code generation or raw coordinate prediction alone, it defines a middle layer through which perception and action become more structured.

3.5 Experimental Setup and Main Results

Agent S is primarily evaluated on OSWorld [27], a benchmark containing 369 tasks across operating-system utilities, office applications, browser tasks, professional software, and multi-app workflows. The paper also evaluates cross-OS transfer on WindowsAgentArena [6]. Table 3.1 summarizes the main OSWorld result.

The headline result is that Agent S with a GPT-4o backbone increases overall success rate from 11.21% to 20.58%, an 83.6% relative improvement over the reported corresponding baseline. The gains are especially strong in daily-use and professional task categories, which is consistent with the paper’s argument that retrieval and hierarchical planning are most useful when tasks depend on procedural knowledge and long workflows. On WindowsAgentArena, Agent S reportedly improves an equivalent setup from 13.3% to

18.2% without explicit adaptation, suggesting some cross-platform generality.

3.6 Ablations and Error Analysis

The paper provides several ablations to support its design claims. Removing web knowledge causes a substantial performance drop, which suggests that application-specific external knowledge remains important. Removing narrative or episodic memory also weakens performance, though to different extents, indicating that the planning and execution layers benefit from distinct experience representations. Additional ablations show that ACI contributes not only to grounding but also to more effective agentic learning.

The error analysis is equally informative. Agent S categorizes failures into planning errors, grounding errors, and execution errors. Execution errors dominate, while grounding errors remain common in several application categories. This finding is important because it qualifies the positive benchmark result: the framework helps, but the system is still far from robust real-world autonomy. Better long-horizon structure does not eliminate the fragility of step-level action policy.

3.7 Assessment

Agent S is a strong framework paper because it does three things well. First, it defines a clear systems problem rather than claiming that scaling alone will solve GUI automation. Second, it introduces modular mechanisms with interpretable roles: web retrieval, hierarchical planning, memory, and ACI. Third, it supports its claims with quantitative gains and nontrivial ablation analysis.

Its limitations are also clear. Absolute success rates remain low for deployment-quality automation. The framework depends on strong proprietary backbones, which reduces the reproducibility of some conclusions. In addition, the memory and retrieval design, though conceptually elegant, does not by itself solve robustness under highly dynamic interfaces. These limitations motivate the next paper: if framework innovation is real but fragile, then open data, tooling, and training infrastructure become the next bottleneck.

3.8 Summary

Agent S is best understood as a framework-level contribution. It demonstrates that hierarchical planning, external knowledge retrieval, memory, and interface abstraction can jointly improve real desktop-task performance, while also exposing the remaining difficulty of the problem through careful ablation and error analysis.

Chapter 4

OpenCUA: Open Foundations for Computer-Use Agents

4.1 Research Problem and Contribution Scope

OpenCUA [22] starts from a broader concern than Agent S. The authors argue that as computer-use agents become more capable, the most influential systems are simultaneously becoming more opaque. Important details about training data, annotation pipelines, reasoning supervision, and model recipes are often closed. This makes it difficult for the research community to reproduce results, understand failure modes, or study social and safety implications in a rigorous way.

OpenCUA responds by contributing an open-source stack for CUA research. The stack includes a cross-OS annotation tool, a large-scale dataset of human demonstrations, an offline benchmark, methods for converting raw demonstrations into trainable trajectories, a reflective long-chain-of-thought supervision recipe, and a suite of trained agent models. As a result, the paper operates at a different layer of the research stack than Agent S: it is less about one framework for inference-time control and more about the open foundations required for systematic progress.

4.2 AgentNet Tool and Dataset Construction

Figure 4.1 presents the overall OpenCUA pipeline. Human interactions are captured by the AgentNet Tool, transformed into trajectories with reasoning and action structure, collected into the AgentNet dataset and benchmark, and then used to train and evaluate agent models.

The data-collection contribution is substantial. AgentNet Tool records screen video, keyboard and mouse signals, and accessibility information in a way intended to interfere

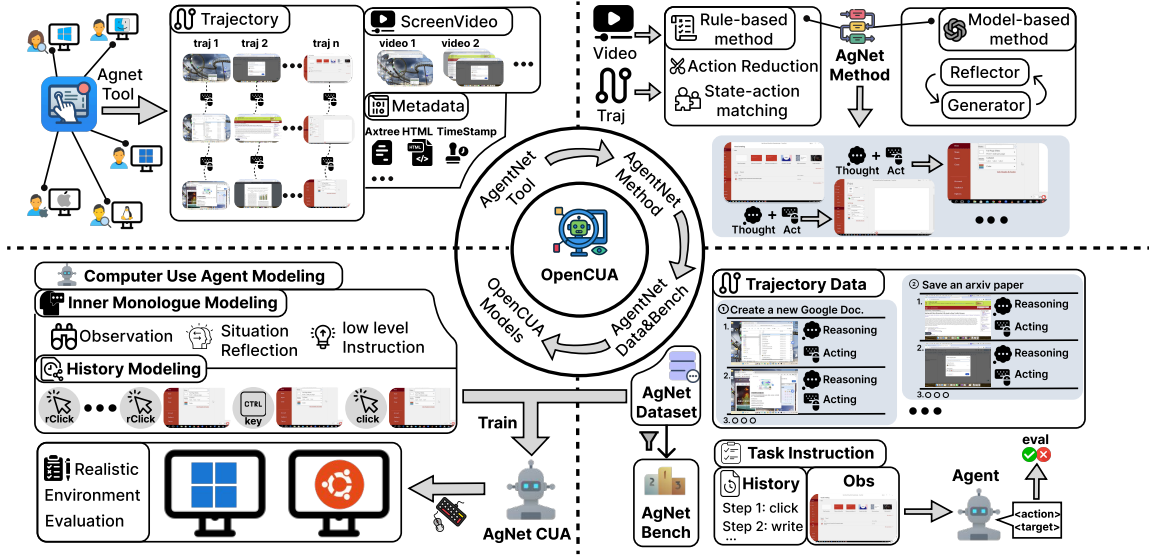


Figure 4.1: Overview of the OpenCUA framework.

minimally with natural user behavior. OpenCUA reports a dataset of roughly 22.6K human-annotated computer-use trajectories across Windows, macOS, and Ubuntu, spanning more than 100 applications and around 200 websites. This scale matters because previous open datasets were often limited to one domain, one platform, or one narrow interaction type.

Just as important as the raw data is the transformation pipeline. Human demonstrations are dense and noisy. A single task may include thousands of low-level input events, many of which are unsuitable for training end-to-end agents directly. OpenCUA therefore applies action reduction and state-action matching to compress these demonstrations into compact trajectories. The paper also emphasizes privacy and verification procedures, which is a meaningful systems consideration for any dataset collected from real desktop usage.

4.3 Reflective Long-CoT Supervision

OpenCUA makes an important methodological claim: simply scaling state-action supervision is insufficient for strong computer-use performance. The paper argues that robust agent behavior depends heavily on richer reasoning traces, especially traces that model planning, reflection, and recovery. This claim is operationalized through a reflective long-chain-of-thought synthesis pipeline.

Figure 4.2 shows this pipeline. A reflector checks step correctness and redundancy, a generator produces structured reasoning components, and a summarizer refines task goals and trajectory-level metadata. The resulting supervision includes multiple reasoning levels, from observation-centered content to action-centered content, with reflection

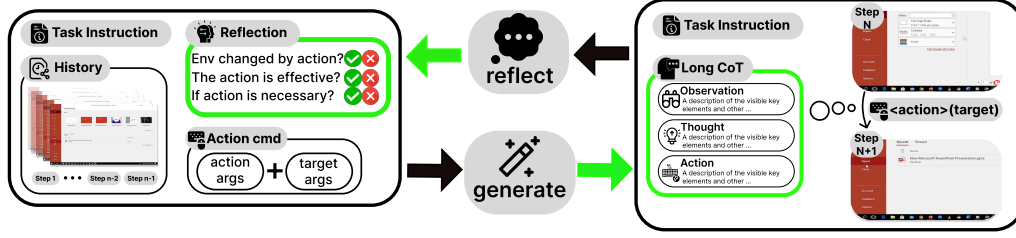


Figure 4.2: Reflective long-CoT synthesis in OpenCUA.

Table 4.1: Selected OpenCUA results on OSWorld-Verified.

Model	15 Steps	50 Steps	100 Steps
UI-TARS-72B-DPO	24.0	25.8	27.1
OpenCUA-7B	24.3	28.1	26.6
OpenCUA-32B	29.7	34.1	34.8
OpenCUA-72B	39.0	44.9	45.0

deliberately emphasized in the middle layer.

This design reflects a broader research trend: agent models benefit when their supervision captures more of the internal structure of competent behavior. In the OpenCUA recipe, context encoding and training data mixtures are also carefully tuned. The model is trained with mixed chain-of-thought formats, multiple screenshots for visual history, grounding data, and general supervised fine-tuning data. The paper treats CUA training not as a simple imitation-learning problem but as a data-engineering and reasoning-representation problem.

The model side of OpenCUA is also notable because it systematically builds on open multimodal backbones such as Qwen2-VL, Qwen2.5-VL, Kimi-VL, and Seed1.5-VL [5, 11, 14, 25]. This supports the authors’ claim that the contribution is not tied to one proprietary model family, but to a transferable recipe for training stronger computer-use agents.

4.4 Empirical Results and Scaling Laws

OpenCUA evaluates its models on online agent benchmarks, offline benchmark settings, and grounding benchmarks. The most visible result is on OSWorld-Verified, where OpenCUA-72B reaches 45.0% success at 100 steps and 44.9% at 50 steps. Table 4.1 shows a condensed version of the reported online results.

These results matter for two reasons. First, they establish that a carefully built open pipeline can produce models that are competitive with the strongest open baselines. Second, the paper reports clear scaling behavior with both more data and larger models. Figure 4.3 illustrates one of the most important messages in the paper: diverse cross-

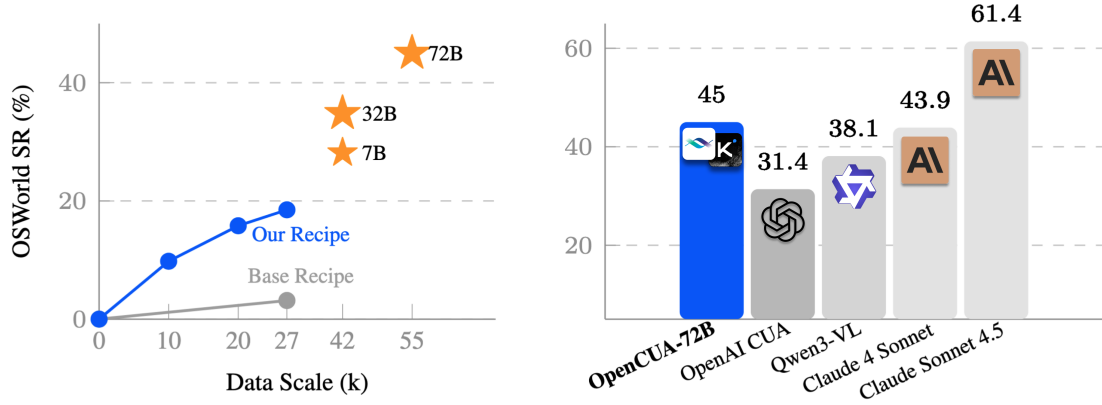


Figure 4.3: Data scaling curves across benchmarks in OpenCUA.

domain data from different operating systems continues to improve performance, rather than causing obvious negative transfer.

The paper also reports strong Pass@ n gains and competitive GUI-grounding results, which together suggest that the OpenCUA recipe improves not only local grounding but also long-horizon decision quality. An important nuance emphasized by the authors is that grounding alone is not enough. Some models remain good at visual grounding but still weak at full task completion, implying that planning and reflective reasoning remain key differentiators.

4.5 Why OpenCUA Matters

OpenCUA changes the research landscape in three ways. First, it reduces dependence on closed data and opaque recipes. Second, it reframes CUA research as an engineering pipeline that includes annotation, processing, supervision design, and evaluation, not only model architecture. Third, it provides evidence that open-source models can make meaningful progress in a domain often dominated by proprietary systems.

At the same time, the paper does not eliminate the core difficulty of the problem. Training cost remains high. Long-horizon brittleness persists even as leaderboard results improve. The data pipeline is much more open than previous efforts, but still difficult for smaller groups to reproduce fully at scale. Most importantly for the logic of this report, OpenCUA still inherits a benchmark regime where GUI interaction remains the primary action model. Once agents start using powerful external tools, a new evaluation problem emerges. This is the gap addressed by OSWorld-MCP.

4.6 Summary

OpenCUA’s importance lies not only in its reported benchmark numbers, but in its re-definition of what serious CUA research infrastructure should include: collection tooling, trajectory processing, reasoning supervision, and open models. It is therefore a foundational systems paper for the area.

Chapter 5

OSWorld-MCP: Benchmarking GUI and Tool Invocation Together

5.1 Research Problem and Benchmark Motivation

OSWorld-MCP [12] is motivated by a fairness problem in contemporary evaluation. Traditional computer-use benchmarks largely assess GUI interaction quality: can the agent click the right thing, type the correct content, and manage a multistep workflow through interface actions? However, modern agents increasingly have access to external tools through protocols such as MCP. When such tools are available, evaluating agents only on GUI interaction becomes incomplete. Worse, comparing a tool-enabled agent against a GUI-only agent under a benchmark that ignores tool usage creates an apples-to-oranges situation.

The paper therefore proposes OSWorld-MCP as a benchmark that jointly measures GUI operation, tool invocation, and hybrid decision making. This is a consequential shift. The benchmark is not only asking whether the agent can operate an interface; it is asking whether the agent can choose the appropriate *action channel* for a given task.

This motivation also distinguishes OSWorld-MCP from recent tool-use benchmarks that are primarily text-centric or server-centric. Benchmarks such as MCPEval, MCP-Radar, LiveMCPBench, and MCP-Bench study important aspects of tool invocation [9, 19, 20, 23], but they do not place the model inside a realistic GUI-based computer environment where every tool choice interacts with visual feedback and interface state.

5.2 From GUI-Only Tasks to Hybrid Action Spaces

Figure 5.1 provides the intuition behind the benchmark. The same task, such as installing a VS Code extension, may require several interface actions in a GUI-only setting

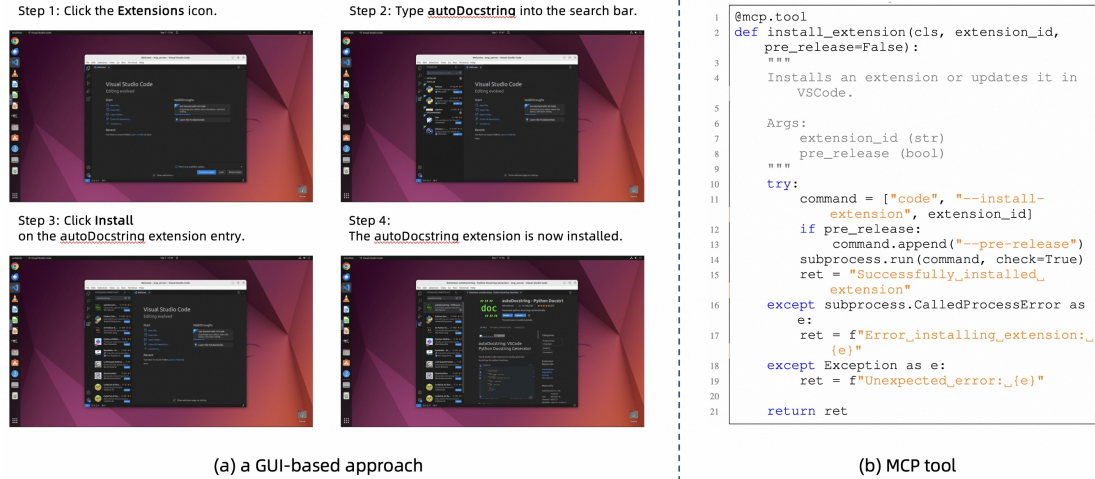


Figure 5.1: A task completed through GUI operations versus MCP tool invocation.

but can be completed more directly through a suitable tool call. The presence of such opportunities makes tool selection itself a meaningful component of competence.

The benchmark is built on top of OSWorld, preserving the real-environment setting while expanding the available action space. At each step, the agent may choose a standard GUI action or invoke an MCP tool. As a result, the benchmark is testing not only execution skill but also high-level decision policy.

5.3 Tool Generation and Benchmark Construction

One of the strongest parts of OSWorld-MCP is its benchmark-construction methodology. The paper does not simply collect an arbitrary set of tools. Instead, it creates an automated pipeline consisting of code generation, code filtering, and MCP wrapping, then combines the resulting tools with curated tools from existing MCP servers. Manual multi-reviewer validation is used to remove duplicates and low-value entries.

Figure 5.2 shows the benchmark framework, while Figure 5.3 shows the tool-generation process. After filtering, the paper reports a final set of 158 high-quality tools covering several common application scenarios.

The quality-control step is particularly important. Many externally available tools are either too task-specific, too trivial, or too redundant for rigorous evaluation. By explicitly analyzing applicability and retaining distractor tools, OSWorld-MCP makes the decision problem more realistic. The benchmark includes both tool-beneficial and non-tool-beneficial tasks, allowing the evaluation to reward correct tool usage and correct tool avoidance.

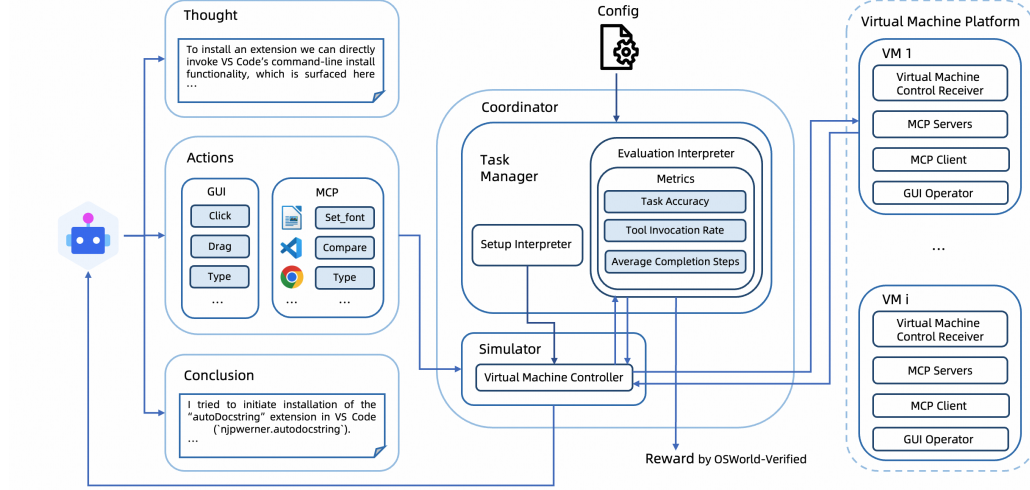


Figure 5.2: Overview of the OSWorld-MCP framework.

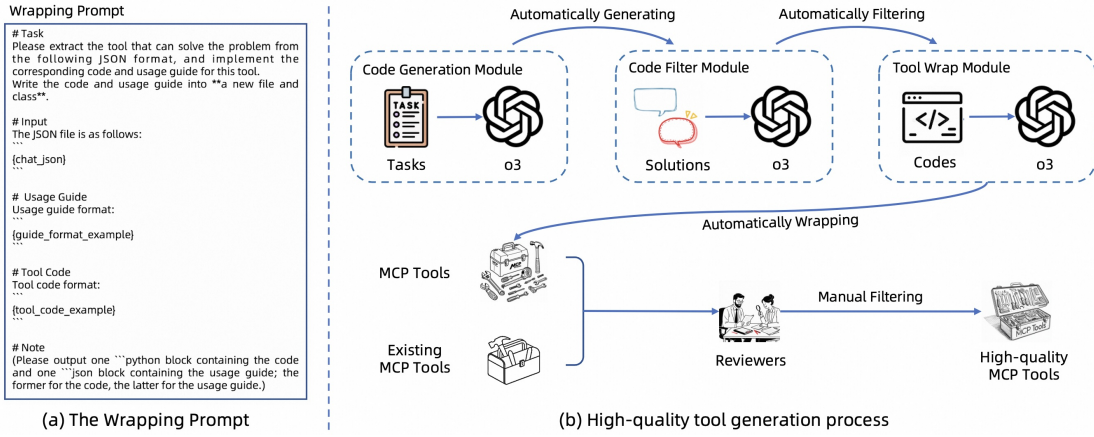


Figure 5.3: Automated tool generation and wrapping in OSWorld-MCP.

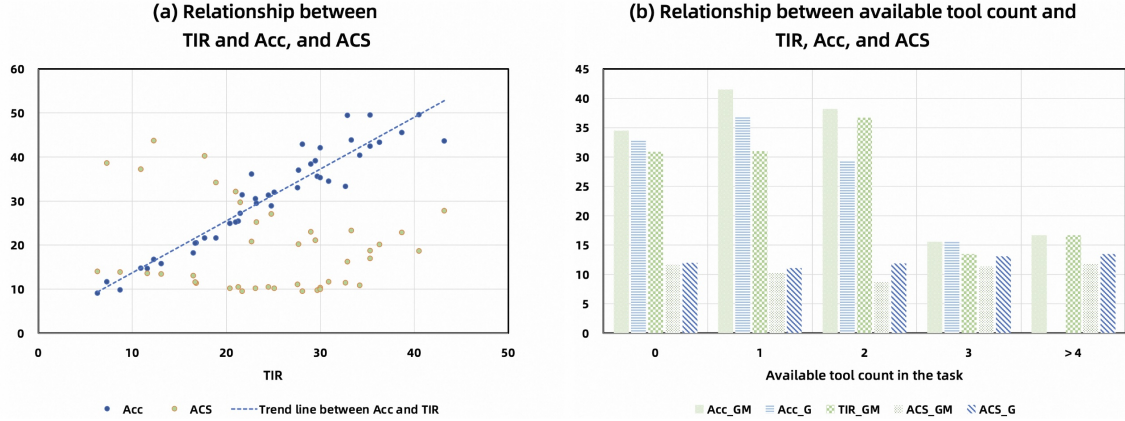
5.4 Metrics: Accuracy, TIR, and ACS

OSWorld-MCP contributes not only a new benchmark environment but also a more diagnostic metric set. Accuracy remains the overall success metric, but the paper adds Tool Invocation Rate (TIR) and Average Completion Steps (ACS). TIR measures the extent to which the agent invokes tools appropriately when tools are beneficial and refrains from invoking them when they are not. ACS captures efficiency in step count.

This metric design is significant because raw accuracy cannot distinguish between qualitatively different policies. An agent may solve some tasks while still relying on inefficient GUI action sequences or invoking tools poorly. TIR and ACS therefore expose new structure in benchmark performance. In effect, OSWorld-MCP argues that once the action space becomes hybrid, evaluation must also become multi-dimensional.

Table 5.1: Selected OSWorld-MCP results.

Model and setting	Accuracy	TIR	ACS
OpenAI o3, GUI only, 15 steps	8.3	—	14.0
OpenAI o3, GUI + MCP, 15 steps	20.4	16.7	11.6
Claude 4 Sonnet, GUI only, 50 steps	40.1	—	24.7
Claude 4 Sonnet, GUI + MCP, 50 steps	43.3	36.3	20.1
Agent-S2.5, GUI + MCP, 50 steps	49.5	35.3	17.0

**Figure 5.4:** Relationships among TIR, accuracy, and average completion steps in OSWorld-MCP.

5.5 Main Results

Table 5.1 summarizes representative results reported in the paper. MCP tools improve success for most evaluated models, but they do so unevenly. For example, OpenAI o3 improves from 8.3% to 20.4% at 15 steps, while Claude 4 Sonnet improves from 40.1% to 43.3% at 50 steps. The strongest reported TIR is only 36.3%, however, showing that even capable models still underuse or misuse tools.

Figure 5.4 further highlights one of the central empirical findings: TIR tends to correlate positively with accuracy, but its relationship with ACS is more complex. Correct tool use can shorten solution paths, but harder tasks may simultaneously require more attempts, more steps, and better policy choice. Thus, efficiency is not a simple linear consequence of tool availability.

5.6 Assessment

OSWorld-MCP makes a crucial contribution by showing that the benchmark layer must evolve when the capability layer evolves. The paper does not propose a fundamentally new agent architecture, but it creates a benchmark that reveals an increasingly important component of agent quality: the ability to choose tools judiciously in real computer

environments.

Its limitations are mainly the limitations of benchmark work. It diagnoses policy quality but does not itself solve tool-selection learning. Some tool-availability annotations and task classifications still depend on manual judgment, which may limit scalability. Moreover, the benchmark inherits the instability and setup complexity common to interactive evaluation. Even so, it fills an important gap. Without such a benchmark, the community would risk overestimating progress by evaluating only GUI action quality while ignoring a rapidly expanding external-tool capability surface.

5.7 Summary

OSWorld-MCP contributes a benchmark-layer correction to the field. Once tool invocation becomes a realistic option for computer-use agents, evaluating GUI behavior alone is no longer enough. The paper formalizes this point and provides concrete metrics for studying it.

Chapter 6

Comparative Synthesis and Future Directions

6.1 From Frameworks to Foundations to Evaluation

The three papers reviewed in this report occupy different layers of the same research stack. Agent S addresses the problem of how to design a more capable interactive system. OpenCUA addresses the problem of how to build open data and model infrastructure for systematic research. OSWorld-MCP addresses the problem of how to evaluate agents fairly when their capabilities extend beyond GUI action sequences.

This layered view clarifies why the papers are complementary rather than redundant. Agent S would be difficult to turn into a community-wide research program without something like OpenCUA. OpenCUA, in turn, would still leave a measurement gap if benchmarks remained limited to GUI-only interaction. OSWorld-MCP completes the picture by adapting the evaluation protocol to a more realistic action space.

6.2 Direct Comparison of the Three Papers

Table 6.1 summarizes the comparison across goals, contribution level, core innovation, and limitations.

6.3 Common Trends

Several common trends emerge across the three papers.

First, the field is moving away from viewing computer use as a flat sequence prediction problem. All three papers, in different ways, treat computer use as structured decision making over long horizons. Agent S does this explicitly with hierarchical plan-

Table 6.1: Comparison of the three reviewed papers.

Aspect	Agent S	OpenCUA	OSWorld-MCP
Primary goal	Improve the control framework for real-OS computer-use agents	Build open data, training, and model foundations for CUAs	Extend evaluation from GUI-only interaction to GUI plus tool invocation
Contribution layer	Inference-time framework design	Open infrastructure and scalable model training	Benchmark and metric design
Core innovation	Experience-augmented hierarchical planning and Agent-Computer Interface	AgentNet data pipeline and reflective long-CoT supervision	Hybrid-action benchmark with MCP tools, TIR, and ACS
Environment focus	OSWorld and WindowsAgentArena	Cross-OS human demonstrations plus multiple benchmarks	OSWorld extended with 158 validated MCP tools
Main empirical message	Structured planning and memory can raise real-task success rates	Open pipelines can scale to competitive open-source performance	Tool availability changes both fairness and interpretation of evaluation
Major strength	Clear modular architecture with meaningful ablations	Full-stack openness and strong scaling evidence	Diagnostic evaluation of hybrid decision making
Main limitation	Low absolute reliability and reliance on strong closed backbones	High cost and incomplete long-horizon robustness	Evaluates tool use but does not directly improve tool-selection policy
Broader field impact	Establishes framework-level design patterns	Makes the field more reproducible and extensible	Redefines what should count as competent computer use

ning. OpenCUA does it through reflective reasoning supervision and evaluation across longer step budgets. OSWorld-MCP does it by exposing the additional strategic layer of action-channel selection.

Second, memory and reflection are becoming core capabilities rather than optional additions. Agent S stores and retrieves prior experience; OpenCUA bakes reflection into the training signal. Both works recognize that successful computer use requires more than local visual grounding. It requires intermediate abstractions about what has been done, what has failed, and what should happen next.

Third, openness is becoming an increasingly important research variable. As the best systems become more consequential, the field needs transparent annotation tools, datasets, and evaluation protocols. OpenCUA addresses this directly, but the need for openness also explains why OSWorld-MCP’s benchmark design work matters and why Agent S’s reliance on strong proprietary backbones remains a meaningful limitation.

6.4 Outstanding Limitations

Despite visible progress, the reviewed papers also reveal persistent limitations.

The first limitation is low absolute reliability. Agent S improves over its baseline significantly, but still solves only around one fifth of OSWorld tasks in its reported setup. OpenCUA’s best open model is much stronger, but 45% success at 100 steps still leaves large room for failure on realistic long-horizon tasks.

The second limitation is fragile grounding and execution behavior. Better planning helps, but local action policies still make incorrect clicks, fail to detect state changes, and waste step budgets in repetitive loops. The error analyses in Agent S and the step-budget observations in OpenCUA both reinforce this point.

The third limitation is that hybrid policies are still immature. OSWorld-MCP shows that tool support helps, yet even strong models have relatively low tool invocation rates. This suggests that current agents often do not recognize when a tool would be beneficial, or they fail to select the right tool when multiple tools are plausible.

The fourth limitation is systems cost. High-quality datasets, reflective supervision, and large-model training all demand substantial compute and human effort. OpenCUA lowers the openness barrier more than the resource barrier.

6.5 Future Directions

The comparative review suggests several promising research directions.

One direction is *better hybrid planning*. Future agents will likely need planners that

reason jointly over GUI actions, external tools, and possibly command-line operations. Such planners should consider not only correctness but also robustness, execution cost, and recovery opportunity.

Another direction is *memory design beyond retrieval*. Current memory modules mostly store textual summaries or subtask exemplars. More structured memory representations, perhaps combining semantic summaries with executable affordance abstractions, may improve transfer and reduce redundancy.

A third direction is *human-centered evaluation*. Current benchmarks emphasize success rate and step efficiency, but real deployment also depends on trust, interpretability, intervention cost, and safety under uncertainty. This is especially important in OS settings where a mistaken action can alter files, settings, or accounts.

Finally, the field needs *continually updated benchmarks*. Operating systems and software ecosystems evolve quickly. A benchmark that is realistic today may become stale tomorrow. OSWorld-MCP already responds to this dynamic by incorporating tools, but future benchmarks may also need collaborative workflows, multi-user contexts, or security-sensitive settings.

6.6 Summary

The comparative synthesis shows that the next phase of CUA research depends on co-evolution across three layers: stronger control frameworks, more open and scalable foundations, and evaluation methods that reflect hybrid action spaces. If any one layer lags behind, the overall field becomes harder to interpret and harder to improve.

Chapter 7

Conclusion

7.1 Summary of Contributions

This report reviewed three papers under a unified theme: computer-use agents in real operating systems. The central claim of the report is that these papers collectively define a coherent research progression. Agent S demonstrates that better framework design can materially improve real-OS task performance through hierarchical planning, experience retrieval, and interface abstraction. OpenCUA shows that long-term progress requires open foundations, including scalable data collection, reflective supervision, and reproducible model recipes. OSWorld-MCP shows that evaluation must evolve in parallel, because tool invocation changes both the action space and the meaning of benchmark success.

Taken together, the papers suggest that CUA research has matured into a full-stack systems problem. The core challenge is no longer just whether a model can click a button. The challenge is whether we can design agents that plan over long horizons, learn from experience, scale through open infrastructure, and be evaluated fairly under realistic hybrid interaction conditions. Much work remains before such agents become reliable enough for broad deployment, but the reviewed papers establish a strong foundation for the next phase of research.

7.2 Future Work

Future work should explore hybrid planners that coordinate GUI actions with tool invocation more explicitly, richer memory abstractions that preserve transferable task structure, human-centered evaluation metrics such as trust and intervention cost, and benchmark packages that remain robust as software ecosystems evolve. These directions would help move computer-use agents from promising prototypes toward dependable operating-system-level assistants.

Bibliography

- [1] Reyna Abhyankar, Qi Qi, and Yiyang Zhang. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. *arXiv preprint arXiv:2506.16042*, 2025.
- [2] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S: An Open Agentic Framework that Uses Computers Like a Human. In *The Thirteenth International Conference on Learning Representations (ICLR 2025), Poster*, 2025.
- [3] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents. *arXiv preprint arXiv:2504.00906*, 2025.
- [4] Anthropic. What Is the Model Context Protocol (MCP)? <https://modelcontextprotocol.io/docs/getting-started/intro>, 2024.
- [5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibor Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL Technical Report. Technical report, 2025.
- [6] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yin-heng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, et al. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. *arXiv preprint arXiv:2409.08264*, 2024.
- [7] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a Generalist Agent for the Web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [8] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? *arXiv preprint arXiv:2403.07718*, 2024.
- [9] Xuanqi Gao, Siyi Xie, Juan Zhai, Shqing Ma, and Chao Shen. MCP-RADAR: A Multi-Dimensional Benchmark for Evaluating Tool Use Capabilities in Large Language Models. *arXiv preprint arXiv:2505.16700*, 2025.

- [10] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.
- [11] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1.5-VL Technical Report. Technical report, 2025.
- [12] Hongrui Jia, Jitong Liao, Xi Zhang, Haiyang Xu, Tianbao Xie, Chaoya Jiang, Ming Yan, Si Liu, Wei Ye, and Fei Huang. OSWorld-MCP: Benchmarking MCP Tool Invocation in Computer-Use Agents. In *The Fourteenth International Conference on Learning Representations (ICLR 2026), Poster*, 2026.
- [13] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. OmniAct: A Dataset and Benchmark for Enabling Multimodal Generalist Autonomous Agents for Desktop and Web. In *European Conference on Computer Vision*, pages 161–178, 2024.
- [14] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-VL Technical Report. Technical report, 2025.
- [15] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [16] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. ComputerRL: Scaling End-to-End Online Reinforcement Learning for Computer Use Agents. *arXiv preprint arXiv:2508.14040*, 2025.
- [17] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. *arXiv preprint arXiv:2504.07981*, 2025.
- [18] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. ShowUI: One Vision-Language-Action Model for GUI Visual Agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [19] Zhiwei Liu, Jieli Qiu, Shiyu Wang, Jianguo Zhang, Zuxin Liu, Roshan Ram, Haolin Chen, Weiran Yao, Shelby Heinecke, Silvio Savarese, et al. MCP Eval: Automatic

- MCP-Based Deep Evaluation for AI Agent Models. *arXiv preprint arXiv:2507.12806*, 2025.
- [20] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. LiveMCPBench: Can Agents Navigate an Ocean of MCP Tools? *arXiv preprint arXiv:2508.01780*, 2025.
- [21] Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A. Rodriguez, Montek Kalsi, Rabiul Awal, Nicolas Chapados, M. Tamer Özsu, Aishwarya Agrawal, David Vazquez, et al. UI-Vision: A Desktop-Centric GUI Benchmark for Visual Perception and Interaction. *arXiv preprint arXiv:2503.15661*, 2025.
- [22] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. OpenCUA: Open Foundations for Computer-Use Agents. In *Advances in Neural Information Processing Systems (NeurIPS 2025), Spotlight*, 2025.
- [23] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, et al. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. *arXiv preprint arXiv:2508.20453*, 2025.
- [24] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv preprint arXiv:2401.16158*, 2024.
- [25] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-VL: Enhancing Vision-Language Models’ Perception of the World at Any Resolution. Technical report, 2024.
- [26] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.
- [27] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *Advances in Neural Information Processing Systems*, 37, 2024.
- [28] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Aguis: Unified Pure Vision Agents for Autonomous GUI Interaction. *arXiv preprint arXiv:2412.04454*, 2024.

- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Vageesh Saxena, Graham Neubig, and Jason Weston. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- [31] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [32] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-Agent-v3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144*, 2025.
- [33] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.